Fix a safety hazard in std::function_ref

Document #:	P3929R0
Date:	2025-11-17
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Jonathan Müller <foonathan@jonathanmueller.dev>

Contents

§ 1 Abstract

Right now, `std::function_ref` can bind to another `std::function_ref` with a different but compatible signature. Unless the underlying function pointer itself is compatible, this creates a reference to the `std::function_ref` object not the underlying function. This is a safety hazard that can make it easy to accidentally create dangling `std::function_ref` objects. We should make this constructor `explicit` to prevent this accidental double-wrapping of `std::function_ref` objects.

§ 2 Motivation

A `std::function_ref` is a function object that stores a reference to another function object. However, because it is itself a function object, unless it is the same exact type and thus can use the copy constructor, it can also store a reference to another `std::function_ref` itself, resulting in a reference to a reference to a function. This is surprising behavior: Other types with reference semantics in the standard library, such as `std::string_view` or

`std::span`, do not have this behavior. It can lead to dangling references, use-after-free, and undefined behavior in various situations.

§ 2.1 Using `std::function_ref` for callbacks that are invoked later

Suppose you're using `std::function_ref` to store some kind of callbacks to be invoked later:

```
void call_me_later(std::function_ref<void()> f);
```

You know that you have to be careful and only pass in function objects that live long enough - it is a reference type after all.

Unsurprisingly, passing in function pointers works fine:

```
void f();
int g(); // we don't care about g's result

call_me_later(f); // okay, stores pointer to `f`
call_me_later(g); // okay, stores pointer to `g`
```

However, if you explicitly construct a `std::function_ref` first you might have a use-after-free:

```
call_me_later(std::function_ref(f)); // okay, copy constructor, stores pointer to `f`
call_me_later(std::function_ref(g)); // use-after-free, stores reference to temporary
        `std::function_ref(g)`!
```

Now why would you do that? Well, you might not do it directly, you just obtain the callback from somewhere else:

```
auto return_g_ptr() { return g; }
auto return_g_ref() { return std::function_ref(g); }

call_me_later(return_g_ptr()); // okay, stores pointer to `g`
call_me_later(return_g_ref()); // use-after-free, stores reference to temporary
        `return_g_ref()`!
```

Similarly, suppose you're having member functions and want to pass them bound to some object:

```
struct foo
{
    void f();
    int g(); // we don't care about g's result
};
foo obj; // suppose obj lives long enough

call_me_later({std::constant_arg<&foo::f>, obj}); // okay, stores bound member function
call_me_later({std::constant_arg<&foo::g>, obj}); // okay, stores bound member function
```

Because it is not at all clear what `{std::constant_arg<&foo::f>, obj}` is supposed to mean, you might want to make it clear that you're explicitly constructing a `std::function_ref`:

```
call_me_later(std::function_ref(std::constant_arg<&foo::f>, obj)); // okay, copy
        constructor, stores bound member function
call_me_later(std::function_ref(std::constant_arg<&foo::g>, obj)); // use-after-free, stores
        reference to temporary `std::function_ref(std::constant_arg<&foo::g>, obj)`!
```

Demo: https://godbolt.org/z/GEhqqjKMv

§ 2.2 Using `std::function_ref` with coroutines or senders/receivers

```
auto result = co_await some_work(std::function_ref(g)); // okay

auto work1 = some_work(std::function_ref(g));
```

```
auto work2 = some_other_work();
co_await ex::when_all(work1, work2); // use-after-free
```

§ 2.3 Using `std::function_ref` in an option struct

Consider the following original code, which is entirely fine:

```
std::function_ref<void(int)> produce() { return some_callback_void; }

void consume(std::function_ref<void(int)> f, int x) { f(x); }

int main() { consume(produce(), 42); }
```

It then evolves to use a configuration struct, here approximated using a `std::pair`:

```
std::function_ref<void(int)> produce() { return some_callback_void; }

void consume(std::pair<std::function_ref<void(int)>, int> o) { o.first(o.second); }

int main() { consume(std::make_pair(produce(), rand() ? 42 : 17)); }
```

It then further evolves to do a more complex construction of the configurations:

```
using O = std::pair<std::function_ref<void(int)>, int>;

std::function_ref<void(int)> produce() { return some_callback_void; }

void consume(O o) { o.first(o.second); }

int main()
{
    auto o = O(produce(), 17);
    if (rand()) o.second = 42;
    consume(o);
}
```

So far, everything is completely fine, but now someone changes the signature of `some_callback_void` to become `some_callback_int`. Ordinarily, this is a backwards compatible change. However, here it leads to a dangling reference:

```
using O = std::pair<std::function_ref<void(int)>, int>;

std::function_ref<int(int)> produce() { return some_callback_int; }

void consume(O o) { o.first(o.second); }

int main()
{
    auto o = O(produce(), 17);
    if (rand()) o.second = 42;
    consume(o); // use after free!
}
```

§ 2.4 Performance

Even in code where everything lives long enough, the (most likely) unintentional double-wrapping is a performance hazard:

```
void do_sth(std::function_ref<void(int)>);

void do_sth_else(std::function_ref<int(int)> f)
{
    do_sth(f);
}
```

`do_sth` receives a reference to a reference to a function, so every time they call it, it involves two indirect calls.

§ 3 Proposal

Right now, `std::function_ref` has two constructors that participate in the construction of a `std::function_ref` from another `std::function_ref`: The implicitly declared copy constructor, which is perfectly fine, and the generic `template<class F> function_ref(F&&)` constructor.

My proposal is to make the latter conditionally `explicit`: If `F` is itself a specialization of `std::function_ref`, the double-wrapping should require an explicit cast and not happen implicitly. This immediately solves all the problems shown above, because now `call_me_later(std::function_ref(g))` is a compiler-error due to the different signature.

While we're at it, we can also resolve [LWG4264] and [NB RU-220] by mandating the requested optimization: Constructing a `std::function_ref` from another one with a compatible bound entity should never do double-wrapping, but always directly bind to the underlying function. For example, constructing a `std::function_ref<void()>` should be constructible from a `std::function_ref<void() const noexcept>` without double-wrapping. In that case, they can also remain implicit conversions.

We can now also enable the assignment operators for compatible function refs, because they will never dangle.

Before

```
void call_me_later(std::function_ref<void()>
        f);

void f();
int g();
void h() noexcept;

call_me_later(f); // okay
call_me_later(g); // okay

call_me_later(std::function_ref(f)); // okay
call_me_later(std::function_ref(g)); // use-
        after-free
call_me_later(std::function_ref(h)); // use-
        after-free

call_me_later(std::function_ref<void()>
        (std::function_ref(g))); // verbose
        use-after-free
call_me_later(std::function_ref<void())>
        (std::function_ref(h))); // verbose
        use-after-free
```

After

```
void call_me_later(std::function_ref<void()>
        f);

void f();
int g();
void h() noexcept;

call_me_later(f); // okay
call_me_later(g); // okay

call_me_later(std::function_ref(f)); // okay
call_me_later(std::function_ref(g)); //
        compiler error, no implicit conversion
call_me_later(std::function_ref(h)); // okay,
        compatible bound entity

call_me_later(std::function_ref<void()>
        (std::function_ref(g))); // verbose
        use-after-free
call_me_later(std::function_ref<void())>
        (std::function_ref(h))); // okay,
        compatible bound entity
```

§ 4 Counterpoints

§ 4.1 Don't use `std::function_ref` if you invoke it later, use `std::move_only_function` instead!

The argument says that writing `call_me_later` taking a `std::function_ref` on its own should not be done, because it is inherently unsafe; types with reference semantics should not be stored somewhere. Just like you should not store a `std::string_view` in a `struct` and use `std::string` instead, you also should not store a `std::function_ref`, but `std::move_only_function` instead. After all, thanks to the small buffer optimization, it does not allocate in most cases anyway.

While this is a good and easy to teach guideline, the argument has three issues:

1. Experts like to get the most performance.

   If you know that all your callbacks are function pointers, it is perfectly reasonable to use `std::function_ref`. After all, `std::move_only_function` has the potential for allocation using a non-customizable allocator. This makes it impossible to use e.g. in embedded code — it isn't even freestanding!

2. Once you have a `std::function_ref`, you're stuck with it.

   ```
   std::move_only_function<void()> callback;

   void do_sth(std::function_ref<void()> f)
   {
       callback = f; // stores a function_ref in the move_only_function!
       callback = std::move_only_function(f); // dito!
   }
   ```

   If all you have is a `std::function_ref` object, you cannot extract the underlying function and store it in a `std::move_only_function`. All you can do, is store a `std::function_ref` itself, which does not help you. You need to refactor everything from the bottom up.

3. `std::function_ref` was explicitly designed to support this use case.

   LEWG overwhelmingly voted to integrate [P2472R3] "Make `std::function_ref` more functional" into `std::function_ref` (8-7-1-0-1 and 3-6-3-0-0). This is the paper that introduced the `std::nontype_t`/`std::constant_arg_t` constructors to allow binding both a member function and `this` in a `std::function_ref`. If `std::function_ref` is only used for parameters which are then immediately invoked, you don't need these constructors at all: You can just pass it a lambda that captures `this`, or use `std::bind_front`, or something like it. The `std::function_ref` will then bind to a temporary, but you're immediately invoking it, so the temporary lives long enough.

   However, LEWG clearly felt that it is useful to support the construction of bound member functions without temporaries, so a `std::function_ref` can be invoked later.

§ 4.2 What about double-wrapping with `std::reference_wrapper` or `third_party::function_ref`?

Constructing a `std::function_ref` from a `std::function_ref` is not the only way to get double-wrapping. It also happens when constructing from a `std::reference_wrapper` or a third-party `function_ref` type:

```
void call_me_later(std::function_ref<void()> f);

call_me_later(std::ref(fn)); // stores reference to reference_wrapper, dangling!
call_me_later(third_party::function_ref(fn));       //       stores       reference       to
        third_party::function_ref, dangling!
```

(This one can also be more obfuscated by having functions return those types.)

While true and unfortunate, it is a different category of problem: When constructing a `std::function_ref` from an arbitrary function object, it is more intuitive that this creates double-wrapping. After all, how would `std::function_ref` know about your `third_party::function_ref` type?

With third-party types you also don't have the problem where refactoring introduces double-wrapping: Your code either starts out with a double-wrapping use-after-free, or it works fine. You don't have the scenario where you change the signature of a function and suddenly you bind a `std::function_ref` to a `std::function_ref`: You will never migrate to a third-party `function_ref` type, but you will migrate to functions with different signatures.

§ 4.3 This doesn't solve all the problems!

Yes, it is still trivial to shoot yourself in the foot with `std::function_ref`. It is a reference type, after all, and because C++ doesn't have a borrow checker, you have to be extremely careful.

However, it solves one particular problem, which is good enough. Users will have the guarantee that constructing a `std::function_ref` from a `std::function_ref` using implicit conversion is always safe, and only have to worry about constructing a `std::function_ref` from another function object.

§ 5 Drawbacks

Making the constructor conditionally `explicit` will make the following code no longer compile:

```
void do_sth(std::function_ref<void(int)>);

void do_sth_else(std::function_ref<int(int)> f)
{
    do_sth(f); // compiler error: no implicit conversion
}
```

Instead, you'd have to cast:

```
void do_sth(std::function_ref<void(int)>);

void do_sth_else(std::function_ref<int(int)> f)
{
    do_sth(std::function_ref<void(int)>(f)); // okay
}
```

However, this also makes it obvious that a double-wrapping is going on, which might be relevant for performance reasons.

Also, if the change ends up being more an annoyance than a benefit, we can always revert the `explicit` later; it would not be a breaking change.

Note that I do not propose making the constructor `explicit` for other function objects; only for incompatible `std::function_ref`.

§ 6 Implementation experience

The changes have been implemented as a patch to [libstdc++] by Tomasz Kamiński and tested locally.

§ 7 Wording

In [func.wrap.ref.class] modify the constructors in the synopsis:

```
// [func.wrap.ref.ctor], constructors and assignment operators
template<class F> function_ref(F*) noexcept;
-template<class F> constexpr function_ref(F&&) noexcept;
+template<class F> constexpr explicit(see below) function_ref(F&&) noexcept;
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
template<auto f, class U> constexpr function_ref(nontype_t<f>, U&&) noexcept;
template<auto f, class T> constexpr function_ref(nontype_t<f>, cv T*) noexcept;

constexpr function_ref(const function_ref&) noexcept = default;
constexpr function_ref& operator=(const function_ref&) noexcept = default;
template<class T> function_ref& operator=(T) = delete;
```

In [func.wrap.ref.ctor] add a new paragraph defining `has-compatible-bound-entity`:

```
template<class... T>
  static constexpr bool is-invocable-using = see below;
```

[…]

```
template<class F>
  static constexpr bool is-compatible-function-ref = see below;
```

? *is-compatible-function-ref*<F> is true, if F denotes specialization of function_ref<R(Args..) *cv2* noexcept(*noex2*)> for some placeholders *cv2* and *noex2*, and:

— is_convertible_v<R(Args...) noexcept(*noex2*), R(Args...) noexcept(*noex*)> is true, and

— is_convertible_v<T *cv*&, T *cv2*&> is true, for some type T.

[*Note:* Another function_ref specialization has a compatible bound entity if the argument and return types are the same, and it has compatible noexcept and cv qualification. — *end note*]

In [func.wrap.ref.ctor] modify paragraphs 5-7:

```
-template<class F> constexpr function_ref(F&& f) noexcept;
+template<class F> constexpr explicit(see below) function_ref(F&& f) noexcept;
```

5 Let T be remove_reference_t<F>.

6 *Constraints:*

(6.1) — remove_cvref_t<F> is not the same type as function_ref,

(6.2) — is_member_pointer_v<T> is false, and

(6.3) — *is-invocable-using*<*cv* T&> is true.

7 *Effects:* If *is-compatible-function-ref*<remove_cv_t<T>> is true, then initializes *bound-entity* and *thunk-ptr* with *bound-entity* and *thunk-ptr* stored by f, respecitvely. Otherwise i~~I~~nitializes *bound-entity* with addressof(f), and *thunk-ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent ([defns.expression.equivalent]) to invoke_r<R>(static_cast<*cv* T&>(f), *call-args*...).

? *Remarks:* The constructor is explicit if remove_cref_t<F> is a specialization of function_ref and *is-compatible-function-ref*<remove_cv_t<F>> is false.

In [func.wrap.ref.ctor] modify paragraph 21:

```
template<class T> function_ref& operator=(T) = delete;
```

21 *Constraints:*

(21.1) — T is not the same type as function_ref,

? — *is-compatible-function-ref*<remove_cv_t<T>> is false,

(21.2) — is_pointer_v<T> is false, and

(21.3) — T is not a specialization of nontype_t.

§ 8 Acknowledgments

Thanks to Tomasz Kamiński for providing feedback, implementation experience, and wording improvements.

§ 9 References

[libstdc++] [RFC] libstdc++: Rework function_ref construction from function_ref [D3929R0].
https://gcc.gnu.org/pipermail/libstdc++/2025-November/064299.html

[LWG4264] Tomasz Kamiński. Skipping indirection is not allowed for function_ref.
https://wg21.link/lwg4264
[NB RU-220] RU-220 22.10.17.01 [func.wrap.general] Adopt LWG4264 Skipping indirection is not allowed for function_ref.
https://github.com/cplusplus/nbballot/issues/790
[P2472R3] Jarrad J. Waterloo, Zhihao Yuan. 2022-05-13. make function_ref more functional.
https://wg21.link/p2472r3